Disclaimer:

Dieses Material ist lizensiert.

Lizenz: CC BY-NC-ND 4.0 (wenn nicht anders angegeben)

- **Attribution (BY)**
- **Non Commercial (NC)**
- **No Derivatives (ND)**
- **No Warranty**
- **No Liability**

Einzelne Beispiele oder seperat geteilte Dokumente könnten einer anderen Lizenz unterliegen.

Autor: © Korobov Viktor, 2026

Netzwerke:

- `ris:LinkedinBox` LinkedIn
- `ris:Github` GitHub
- `ris:Youtube` YouTube
- `ris:CodeS` LeetCode
- `ris:Steam` Steam

WIP:

Dieses Kursmaterial ist ein lebendiges Dokument und wird mit einem KI Assisstenten gewartet.

Letzter Stand: 04.11.26.

Gebt aufmerksam Acht auf das Gesagte!

Unterstützung:

Bei Anmerkung zum Material kontakiert mich gerne unter:

- viktor.korobov@edu.korobov-solutions.de

Unterstützt mich mit Feedback unter:

- Fragebogen

Joins & Views

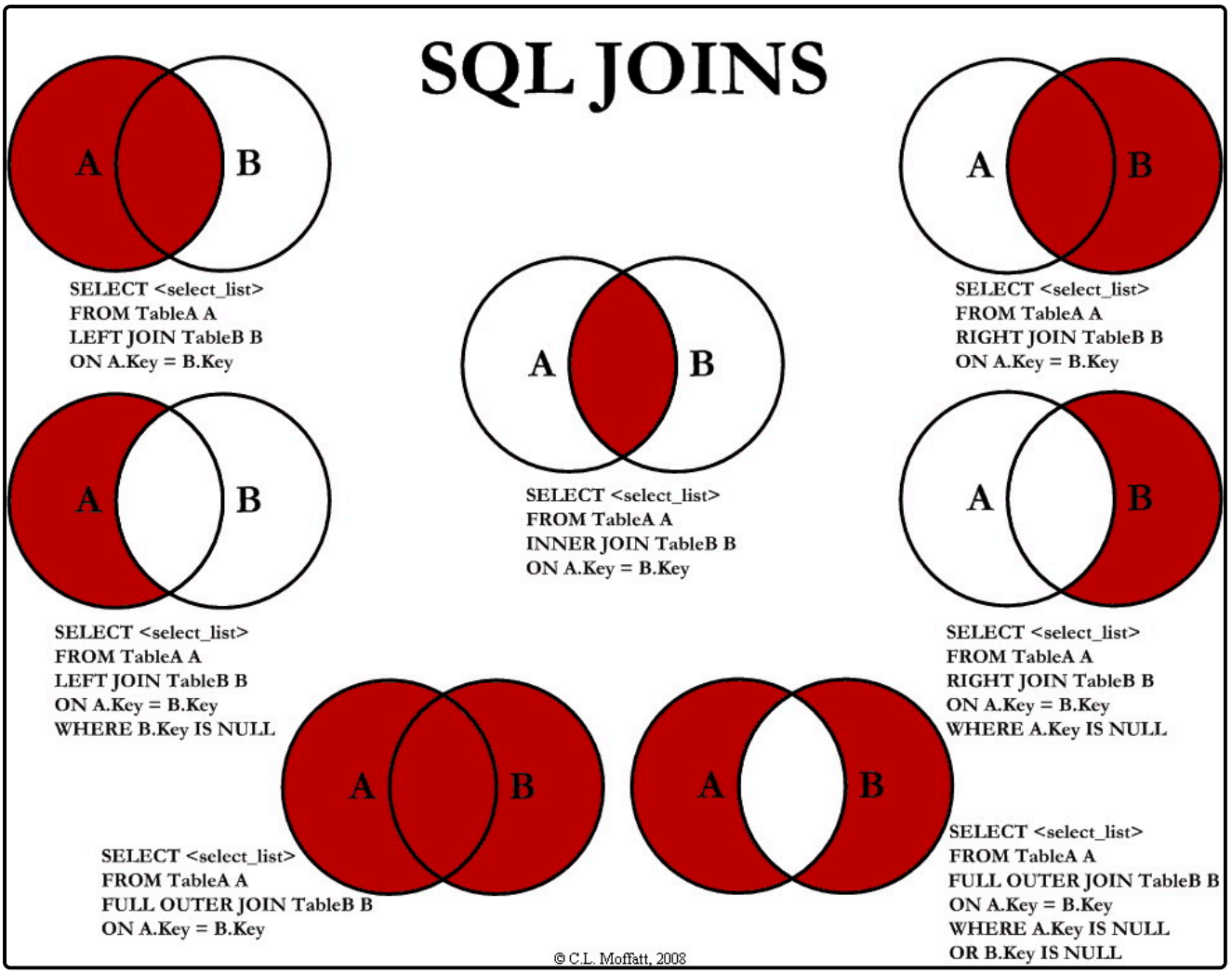
Joins

Ein `JOIN` in SQL verknüpft Daten aus mehreren Tabellen anhand einer definierten Beziehungsbedingun (meist `Fremdschlüssel = Primärschlüssel`). So können Informationen kombiniert werden, die auf mehrere Tabellen verteilt sind.

Joins sind notwendig, weil in relationalen Datenbanken Daten normalisiert vorliegen, also über mehrere logisch getrennte Tabellen verteilt sind.

Übersicht der `JOIN`-Typen

`JOIN`-Typ	Beschreibung
`INNER JOIN`	Nur Datensätze mit Übereinstimmungen in beiden Tabellen
`LEFT JOIN`	Alle Datensätze der linken Tabelle, auch ohne Übereinstimmung
`RIGHT JOIN`	Alle Datensätze der rechten Tabelle, auch ohne Übereinstimmung
`FULL OUTER JOIN`	Alle Datensätze beider Tabellen – nicht in MariaDB direkt verfügbar
`CROSS JOIN`	Kartesisches Produkt: jede Zeile von A mit jeder Zeile von B
`SELF JOIN`	Verknüpft eine Tabelle mit sich selbst



Tabellenbeschreibung für Beispiele

```
Tabelle: Mitarbeiter
+----+-------------+-------------+
| ID | Name        | Abt_ID      |
+----+-------------+-------------+
| 1  | Alice       | 10          |
| 2  | Bob         | 20          |
| 3  | Charlie     | NULL        |
+----+-------------+-------------+

Tabelle: Abteilung
+----+-------------------+
| ID | Bezeichnung       |
```

```
+----+------------------+
| 10 | IT               |
| 20 | Controlling      |
| 30 | Personal         |
+----+------------------+
```

Beispiel (`INNER JOIN`):


```sql
SELECT Mitarbeiter.Name, Abteilung.Bezeichnung
FROM Mitarbeiter
INNER JOIN Abteilung ON Mitarbeiter.Abt_ID = Abteilung.ID;
```

Beispielhafter Output:



```
+----------+-------------+
| Name     | Bezeichnung |
+----------+-------------+
| Alice    | IT          |
| Bob      | Controlling |
+----------+-------------+
```

Beispiel (`LEFT JOIN`):


```sql
SELECT Mitarbeiter.Name, Abteilung.Bezeichnung
FROM Mitarbeiter
LEFT JOIN Abteilung ON Mitarbeiter.Abt_ID = Abteilung.ID;
```

Beispielhafter Output:



```
+----------+-------------+
| Name     | Bezeichnung |
+----------+-------------+
| Alice    | IT          |
| Bob      | Controlling |
| Charlie  | NULL        |
+----------+-------------+
```

Beispiel (`RIGHT JOIN`):


```sql
SELECT Mitarbeiter.Name, Abteilung.Bezeichnung
FROM Mitarbeiter
RIGHT JOIN Abteilung ON Mitarbeiter.Abt_ID = Abteilung.ID;
```

Beispielhafter Output:



```
+----------+-------------+
| Name     | Bezeichnung |
+----------+-------------+
| Alice    | IT          |
| Bob      | Controlling |
| NULL     | Personal    |
+----------+-------------+
```

Beispiel (`FULL OUTER JOIN`):


```sql
-- LEFT JOIN: alle Mitarbeiter + passende Abteilungen
SELECT Mitarbeiter.Name, Abteilung.Bezeichnung
FROM Mitarbeiter
LEFT JOIN Abteilung ON Mitarbeiter.Abt_ID = Abteilung.ID

UNION

-- RIGHT JOIN: alle Abteilungen + passende Mitarbeiter
SELECT Mitarbeiter.Name, Abteilung.Bezeichnung
FROM Mitarbeiter
RIGHT JOIN Abteilung ON Mitarbeiter.Abt_ID = Abteilung.ID;
```

- Der `LEFT JOIN` liefert alle Zeilen aus `Mitarbeiter`, auch wenn es keine passende Abteilung gibt.
- Der `RIGHT JOIN` liefert alle Zeilen aus `Abteilung`, auch wenn kein Mitarbeiter zugeordnet ist.
- Das `UNION` kombiniert beide Ergebnismengen und entfernt automatisch Duplikate.

> **Tipp:**
>
> MariaDB unterstützt keinen eigenen `FULL OUTER JOIN`.
> Um dennoch Daten aus beiden Tabellen vollständig zu kombinieren muss eine Kombination aus `LEFT JOIN` und `RIGHT JOIN` über `UNION` verwendet werden.

Beispielhafter Output:



```
+----------+-------------+
| Name     | Bezeichnung |
+----------+-------------+
| Alice    | IT          |
| Bob      | Controlling |
| Charlie  | NULL        |
| NULL     | Personal    |
+----------+-------------+
```

Beispiel (`CROSS JOIN`):


```sql
SELECT Mitarbeiter.Name, Abteilung.Bezeichnung
FROM Mitarbeiter
CROSS JOIN Abteilung;
```

Beispielhafter Output:

```
+----------+-------------+
| Name     | Bezeichnung |
+----------+-------------+
| Alice    | IT          |
| Alice    | Personal    |
| Alice    | Controlling |
| Bob      | IT          |
| Bob      | Personal    |
| Bob      | Controlling |
| Charlie  | IT          |
| Charlie  | Personal    |
| Charlie  | Controlling |
+----------+-------------+
```

Beispiel (`SELF JOIN`):

```sql
SELECT a.Name AS MA1, b.Name AS MA2
FROM Mitarbeiter a
JOIN Mitarbeiter b ON a.Abt_ID = b.Abt_ID
WHERE a.ID < b.ID;
```

Beispielhafter Output:

```
+--------+--------+
| MA1    | MA2    |
+--------+--------+
| Alice  | Bob    |
+--------+--------+
```

Views

Eine View ist eine virtuelle Tabelle, die auf dem Ergebnis einer gespeicherten `SELECT`-Abfrage basiert.
Sie enthält keine eigenen Daten, sondern bezieht sich dynamisch auf bestehende Tabellen.

Hauptmerkmale einer View:

- Sie erscheint wie eine normale Tabelle
- Sie kann in anderen Abfragen genutzt werden
- Sie dient der Abstraktion, Wiederverwendbarkeit und Sicherheit
- Änderungen an zugrundeliegenden Tabellen wirken sich sofort auf die View aus

Beispiel (`VIEW` -Blueprint):

```sql
CREATE VIEW viewname AS
```

```
SELECT ...
FROM ...
WHERE ...;
```

Beispiel (`VIEW`):


```
CREATE VIEW aktive_teilnehmer AS
SELECT p.Vorname, p.Nachname, t.Geburtsdatum
FROM Teilnehmer t, Person p
WHERE t.Person_ID = p.ID;
```



```
SELECT * FROM aktive_teilnehmer WHERE YEAR(Geburtsdatum) < 2010;
```

Danksagung:

Liebe Lernende, vielen Dank für die Aufmerksamkeit!

Liebe Veranstalter, vielen Dank für die Bühne!

Unterstützung:

Bei Anmerkung zum Material kontakiert mich gerne unter:

- viktor.korobov@edu.korobov-solutions.de

Unterstützt mich mit Feedback unter:

- Fragebogen